

Supervised Fine-Tuning

pig7selene

September 5, 2026

Abstract

Supervised Fine-Tuning adapts a pretrained decoder-only language model to instruction following by training on demonstrations of desired responses. This chapter shows that SFT retains the autoregressive next-token machinery of pretraining while changing the data distribution, conversation serialization, and set of target tokens. It develops assistant-only loss masking for multi-turn conversations, treats Chat Templates as versioned model-data contracts, examines packing and data-mixture choices, introduces full-parameter adaptation and LoRA, and concludes with failure modes, evaluation criteria, and implementation contracts.

1 Introduction

Pretraining produces a model that assigns probabilities to text continuations. That capability is broad, but the pretraining distribution does not uniquely specify how the model should respond when a user asks a question, requests a transformation, or supplies a system instruction. A web corpus contains answers, questions, dialogue fragments, quoted conversations, unfinished pages, and many other continuation patterns. Even a strong base model can therefore continue an instruction instead of answering it, imitate both sides of a dialogue, or choose a response style that is plausible under the corpus but unsuitable for an assistant.

Post-training narrows this behavioral ambiguity. Its first major stage is commonly **Supervised Fine-Tuning** (SFT): continue optimizing a pretrained model on examples that pair an instruction or conversational context with a desired assistant response. Early instruction-tuning work demonstrated that training across many tasks expressed through natural-language instructions can improve generalization to unseen tasks [1]. InstructGPT likewise began its post-training pipeline with supervised demonstrations before introducing preference-based stages [2].

This chapter isolates SFT. It does not introduce Reward Models, RLHF, PPO, DPO, GRPO, or Reasoning RL. The central question is more basic: what statistical problem is solved when a pretrained Causal Language Model is trained to imitate high-quality instruction-response demonstrations?

2 From Pretraining to Instruction Following

Chapter 5 defined pretraining as maximum likelihood over token sequences drawn from a broad corpus distribution. SFT normally retains the same decoder, language-model head, causal Attention Mask, teacher-forced inputs, and token-level cross-entropy. What changes is the distribution of sequences presented to the model and, often, the subset of their tokens that contributes to the loss.

Let $\mathcal{D}_{\text{pre}}$ denote the effective pretraining distribution and $\mathcal{D}_{\text{SFT}}$ the distribution induced by curated demonstrations. Pretraining estimates conditionals that are useful throughout $\mathcal{D}_{\text{pre}}$. SFT places additional probability mass on sequences in which a structured prompt is followed by a desired response. The optimization objective remains next-token prediction, but the observed conditional patterns now repeatedly associate instructions and role markers with assistant behavior.

This distinction separates an **objective** from the behavior induced by its data. Next-token prediction is the local likelihood objective in both stages. Instruction following is not a different output head or a symbolic execution rule added to the model; it is behavior learned from the conditional distribution represented by SFT examples. The model is shown that after a serialized system-and-user context,

certain response tokens should receive high probability. Data coverage, formatting, target selection, and weighting therefore determine which notion of “following” the model learns.

SFT should not be described as supplying all knowledge to an otherwise empty model. The pretrained parameters already encode linguistic and factual regularities; SFT teaches the model how to expose and organize those capabilities under a new interaction protocol. LIMA provides a deliberately narrow empirical illustration: a strong base model acquired substantial response behavior from only 1,000 carefully curated demonstrations [3]. That result does not imply that every model or domain requires little data, but it makes clear that example quality and the pretrained starting point are inseparable from SFT scale.

3 Supervised Fine-Tuning Objective

An SFT record begins as structured data rather than as a token tensor. For record n , let

$$c^n = (q_1^n, q_2^n, \dots, q_{J_n}^n) \quad (1)$$

be an ordered sequence of J_n messages. Each message $q_j = (r_j, u_j)$ has a role r_j , such as system, user, or assistant, and content u_j . A deterministic serialization function φ applies a Chat Template, and the tokenizer τ then produces

$$x^n = \tau(\varphi(c^n)) = (x_1^n, \dots, x_{T_n}^n). \quad (2)$$

The message structure has disappeared by the time the Transformer receives x^n ; the model operates on one token sequence. The preprocessing pipeline must therefore preserve a parallel record of which token positions came from which message spans.

For M serialized records, let $m_t^n \in \{0, 1\}$ indicate whether the target token x_{t+1}^n is supervised. With $N = \sum_n \sum_t m_t^n$ valid targets, the token-averaged SFT loss is

$$\mathcal{L}_{\text{SFT}(\theta)} = -\frac{1}{N} \sum_{n=1}^M \sum_{t=1}^{T_n-1} m_t^n \log p_{\theta}(x_{t+1}^n | x_{1:t}^n). \quad (3)$$

Equation 3 is the same masked autoregressive negative log-likelihood derived in Chapter 5. Its meaning changes through $\mathcal{D}_{\text{SFT}}$, φ , and m . The Causal Attention Mask still determines what each position may read; the loss mask determines which next-token predictions influence the parameter update. Confusing these masks can either leak future information or optimize unintended targets.

3.1 Full-Sequence and Assistant-Only Loss

Under **full-sequence loss**, every eligible non-padding token in the serialized conversation contributes to Equation 3. The model is trained to predict system text, user text, role markers, assistant text, and boundaries. This can be appropriate when every span is regarded as modeled data, but it spends gradient weight reproducing prompts that the runtime normally supplies.

Under **assistant-only loss**, prompt targets are masked out and assistant response targets are retained. The system and user tokens remain in the input context, so they still affect assistant hidden states and gradients through the computation graph; they simply do not create direct token-level errors of their own. Llama 2 reports zeroing the loss on user-prompt tokens and backpropagating only on answer tokens during its SFT stage [4].

Consider the multi-turn role sequence

$$\text{System} \rightarrow \text{User} \rightarrow \text{Assistant} \rightarrow \text{User} \rightarrow \text{Assistant}. \quad (4)$$

With assistant-only supervision, targets inside the system message and both user messages receive $m_t = 0$. Targets inside both assistant responses receive $m_t = 1$. The end-of-message token following an assistant response is usually supervised because the model must learn when to stop that turn. Whether the opening assistant-role marker is supervised depends on the inference protocol: if the runtime appends that marker as a generation prompt, it is context and may be masked; if the model must generate it, it is a target. The policy must be defined in token coordinates after serialization, not inferred later from decoded text.

Serialized span	Context	Typical target mask	Reason
System content	Yes	0	Conditions the response but is normally supplied by the application.
User content	Yes	0	Defines the request; reproducing it is not the assistant task.
Assistant content	Yes	1	Provides the demonstrated response behavior.
Assistant turn ending	Yes	1	Teaches the model to terminate or hand back control.
Padding	No	0	Carries neither semantic context nor a valid target.

Table 1: A common assistant-only policy for a serialized conversation. Role and boundary-token treatment remains template-specific and must be declared explicitly.

Table 1 is a policy, not a universal convention. Some instruction-tuning datasets use full-sequence loss; some supervise selected role or boundary tokens; some supervise only the final assistant turn. These objectives assign different weights to the same stored conversation. A reported “SFT loss” is therefore incomplete without its serialization and masking rules.

4 Conversations as a Model-Data Interface

4.1 Chat Templates, Roles, and Boundaries

A Chat Template is the executable definition of φ in Equation 2. It specifies how messages are ordered, how roles are marked, which separators and end-of-turn tokens are inserted, how an absent system message is handled, and what prefix begins an assistant generation. Official Transformers documentation emphasizes that chat models still consume token sequences and that different models may require different control tokens even when they share a base architecture [5].

The template is therefore a model-data interface contract, not a cosmetic string formatter. If training serializes a user message as user followed by one boundary convention but inference supplies a different marker or duplicates beginning- and end-of-sequence tokens, the runtime presents prefixes outside the learned protocol. The model may answer in the wrong role, expose separators, fail to stop, or continue the user turn. Tokenizer version, special-token IDs, template source, whitespace behavior, and the generation-prompt convention must be versioned with the checkpoint.

Special tokens do not possess role semantics merely because they have suggestive names. Their embeddings and conditional effects are learned from their positions in training sequences. A reserved assistant marker becomes meaningful because the corpus repeatedly places it at a particular transition. Message boundaries similarly teach turn termination only if their target treatment is consistent. Chapter 1’s tokenizer contract therefore extends directly into SFT: a template can refer only to tokens whose identities and segmentation behavior are stable.

4.2 Single-Turn and Multi-Turn Supervision

A single-turn example contains one prompt and one response. Its supervision teaches a direct conditional mapping and makes example boundaries simple. A multi-turn example contains alternating messages whose later responses depend on earlier context. It can teach reference resolution, correction, constraint persistence, and the convention that the assistant should respond only after the user has yielded the turn.

Multi-turn training also introduces weighting choices. If every assistant span is supervised, one long conversation contributes several response targets and may dominate a token-averaged mixture. If only the final response is supervised, earlier assistant messages remain teacher-forced context but do not receive direct loss. Neither policy is automatically correct. The chosen unit of sampling, the number of supervised turns, and the valid-token denominator determine how much weight a conversation receives.

Teacher forcing creates a further distinction between training and deployment. During training, the second assistant response is conditioned on the recorded first assistant response. At inference, it is conditioned on the model’s own earlier output. SFT does not remove this exposure difference. Diverse multi-turn data and evaluations with model-generated history can reveal it, but changing that limitation requires methods beyond ordinary supervised likelihood.

5 Sequence Construction and Data Mixtures

5.1 Packing and Conversation Boundaries

SFT examples are often much shorter than the model context, so packing concatenates several serialized conversations into one fixed-length tensor. As in Chapter 6, packing improves token utilization but creates boundary semantics. A later conversation must not accidentally inherit an earlier conversation as its prompt unless that stream interpretation is intentional. Pipelines can isolate samples with a block-diagonal Attention Mask or accept cross-sample attention while inserting explicit boundaries; the decision changes the conditioning distribution.

Loss alignment must survive packing independently of attention. The first token of a new packed conversation should not become an unintended target of the preceding conversation. Assistant-only masks must be concatenated with exactly the same offsets as token IDs, and padding, truncation, and end-of-turn insertion must preserve N in Equation 3. When truncation cuts an assistant response, the retained fragment may teach a response without its proper termination; when it removes the prompt but retains targets, it creates an invalid example. Segment-aware tests are therefore essential.

5.2 Mixtures, Sampling, and Curriculum

An SFT corpus commonly combines task demonstrations, open-ended dialogue, reasoning traces, code, safety behavior, domain-specific records, and synthetic responses. If component k induces distribution P_k and is sampled with weight π_k , the effective demonstration distribution is

$$P_{\text{SFT}(c)} = \sum_{k=1}^K \pi_k P_{k(c)}, \quad \pi_k \geq 0, \quad \sum_{k=1}^K \pi_k = 1. \quad (5)$$

The mixture weights determine which response patterns receive repeated likelihood pressure. Raw example counts are not realized weights: conversation lengths, assistant-only masks, rejection rates, packing, and resampling change the share of supervised tokens. The FLAN Collection found task balancing and enrichment choices to be consequential in instruction tuning [6]. A reliable pipeline therefore reports both sampling probabilities and realized assistant-target-token fractions.

A curriculum makes π_k or example difficulty depend on training progress. It may begin with clean, direct demonstrations and later increase complex or specialized examples, or it may interleave all components from the start. Curriculum is not intrinsically superior to static mixing: changing the order changes optimizer history, and a late narrow phase can overwrite broad behavior. The schedule should be treated as an experimental variable, indexed by updates or valid target tokens, rather than as undocumented loader behavior.

6 Data Quality, Diversity, and Failure Modes

SFT datasets are small relative to pretraining corpora, so each repeated pattern can exert substantial influence. Exact or near-duplicate instructions increase the effective weight of their targets and can make validation optimistic if duplicates cross the split. Low-quality synthetic responses can transfer factual errors, verbosity, refusal habits, and formatting artifacts from the generator. Self-Instruct illustrates both the value of generated instruction data and the need to filter invalid or similar generations before fine-tuning [7].

Conflicting supervision occurs when materially equivalent prompts receive incompatible answers, policies, or styles without contextual features that explain the difference. Maximum likelihood must

distribute probability across those targets; it cannot infer a hidden annotation policy. Excessive boilerplate can likewise teach the model to emit preambles or markdown scaffolding regardless of the request. A narrow domain mixture can improve in-domain imitation while reducing breadth, and repeated responses with one rhetorical shape can produce **response-style collapse**: outputs remain grammatical but converge toward a small family of lengths, openings, and structures.

Quality and diversity are therefore joint requirements. Quality asks whether a target is correct, relevant, self-contained, and compatible with the declared policy. Diversity asks whether the corpus covers distinct tasks, domains, languages, response lengths, interaction patterns, and acceptable styles without relying on superficial paraphrase counts. More examples help only when their marginal supervision is useful. LIMA’s curated-data result and FLAN’s mixture ablations should be read as complementary evidence: careful examples matter, but coverage and weighting still determine generalization [3], [6].

Two familiar optimization failures acquire a post-training form. **Overfitting** appears when training loss continues to improve while held-out instruction performance, calibration, or response diversity deteriorates. **Catastrophic forgetting** denotes degradation of capabilities retained in the base model as updates specialize the parameters to the SFT distribution. Excessive epochs, a high learning rate, duplicated data, and an overly narrow mixture can intensify both risks. Mitigations include early checkpoint comparison, lower update magnitude, broader rehearsal data, explicit capability-regression suites, and parameter-efficient adaptation; none substitutes for a protected evaluation set.

7 Full-Parameter and Parameter-Efficient Adaptation

Full-parameter fine-tuning updates every trainable parameter θ under Equation 3. It gives the optimizer maximal freedom to reshape the model and is a direct continuation of pretraining optimization. Its cost is also direct: gradients and optimizer states are maintained for the full model, each adapted checkpoint stores a full parameter set, and updates can alter broadly useful representations.

Parameter-Efficient Fine-Tuning (PEFT) freezes most pretrained parameters and learns a smaller adaptation state. PEFT reduces trainable-state memory and makes it practical to store multiple task or domain adaptations. It also constrains the update subspace, which can be beneficial regularization but can limit adaptation when the chosen modules or capacity are insufficient. PEFT changes which parameters are optimized, not the definition of the token-level SFT objective.

7.1 LoRA as a Low-Rank Update

Low-Rank Adaptation (LoRA) represents the update to a selected weight matrix with two thin trainable matrices [8]. For a frozen pretrained projection $W_0 \in R^{d_{\text{out}} \times d_{\text{in}}}$, write

$$W' = W_0 + \Delta W, \quad \Delta W = BA, \quad (6)$$

where $A \in R^{r \times d_{\text{in}}}$, $B \in R^{d_{\text{out}} \times r}$, and $r \ll \min(d_{\text{in}}, d_{\text{out}})$. Because $\text{rank}(BA) \leq r$, the learned change is restricted to a low-rank subspace. The trainable parameter count for this projection is

$$r(d_{\text{in}} + d_{\text{out}}) \quad \text{rather than} \quad d_{\text{in}}d_{\text{out}}. \quad (7)$$

Implementations commonly scale the update by $\frac{\alpha}{r}$ and initialize one factor so that $\Delta W = 0$ at the start, preserving the base function before training. During the forward pass, the layer computes $W_0x + B(Ax)$; after training, the update can be stored separately or merged into W_0 for deployment. Rank, scaling, target modules, bias treatment, and merge convention are part of the adapter specification. This chapter stops at that contract; other adapter, prefix, and prompt-tuning methods belong to a broader PEFT treatment.

8 Evaluating an SFT Model

Held-out SFT loss measures likelihood under a fixed serialization and target mask. It is useful for detecting overfitting and comparing checkpoints within one protocol, but it does not by itself establish

instruction following. A model can assign high likelihood to reference wording while failing semantically equivalent prompts, or produce a valid answer different from the single reference.

Evaluation should therefore separate several questions. Task suites measure correctness under explicit answer extraction or execution rules. Format tests measure whether requested schemas, role boundaries, and stopping behavior are obeyed. Human evaluation can assess open-ended helpfulness and clarity when deterministic reference metrics are inadequate. Capability-regression suites compare the tuned model with its base checkpoint on retained knowledge and skills. Safety evaluation belongs in the release process even when safety alignment is not the objective of this chapter.

The evaluation distribution must be protected from duplicate or paraphrased training examples, and decoding settings must be fixed when comparing checkpoints. Single-turn and multi-turn behavior should be evaluated separately; the latter should include model-generated history rather than only gold prior responses. Chapter 11’s distinction between validation loss and downstream behavior remains intact: one monitors the declared likelihood objective, while the other tests whether the learned conditional distribution produces the intended interaction.

9 Implementation Contracts

The data contract should preserve the original message sequence, source identity, quality decisions, and split assignment. It should version the Chat Template, tokenizer, special-token IDs, generation-prompt convention, and exact mapping from message spans to token spans. Preprocessing must reject or explicitly handle unsupported roles, empty assistant targets, duplicated boundary tokens, truncated prompts, and conversations whose supervised-token count is zero.

The tensor contract should expose input IDs, shifted targets, causal or segment-aware Attention Masks, loss masks, segment IDs where used, and the valid-target denominator. Tests should serialize hand-written single- and multi-turn conversations and verify every m_t against the intended role span, including assistant opening and closing markers. Packing tests should prove that token IDs, masks, and segment offsets remain aligned and that one sample cannot create an unintended target in another. The optimization contract should record the base checkpoint, trainable parameter set, full-parameter or PEFT mode, optimizer and schedule, effective batch definition, gradient accumulation, precision policy, and number of valid assistant tokens consumed. A LoRA artifact additionally requires rank, scaling, target-module names, factor orientation, initialization, dtype, bias policy, and merge status. Checkpoint evaluation should retain the template and masking metadata needed to reproduce both validation loss and generation.

Finally, the inference contract must apply the same serialization semantics used in training. It should distinguish formatting a completed training conversation from appending a generation prompt, prevent duplicate special tokens, and specify which stop tokens or message boundaries terminate the assistant. A model checkpoint without this interface metadata is not a complete chat model artifact.

10 Summary

Supervised Fine-Tuning is a distributional adaptation of a pretrained Causal Language Model. The architecture and next-token likelihood machinery remain largely unchanged, while instruction-response demonstrations, Chat Templates, and loss masks define a new conditional learning problem. Full-sequence loss models every eligible serialized token; assistant-only loss uses system and user messages as context while assigning direct supervision to selected assistant targets. Multi-turn conversations, packing, and truncation make these token-level boundaries operational rather than merely conceptual.

SFT quality depends on more than example count. Mixture weights determine which behaviors are emphasized; duplicated, synthetic, conflicting, or stylistically narrow targets can distort the learned response distribution. Full-parameter fine-tuning offers unconstrained adaptation at full training-state cost, whereas LoRA learns a low-rank update to selected frozen projections. In either case, held-out

loss must be complemented by instruction, format, multi-turn, and capability-regression evaluation. The resulting model is reproducible only when its tokenizer, Chat Template, special tokens, loss policy, packing rules, adaptation state, and inference protocol are treated as one versioned contract.

References

- [1] J. Wei *et al.*, “Finetuned Language Models Are Zero-Shot Learners,” in *International Conference on Learning Representations*, 2022. [Online]. Available: <https://openreview.net/forum?id=gEZrGCozdqR>
- [2] L. Ouyang *et al.*, “Training Language Models to Follow Instructions with Human Feedback,” in *Advances in Neural Information Processing Systems 35*, 2022, pp. 27730–27744. doi: 10.52202/068431-2011.
- [3] C. Zhou *et al.*, “LIMA: Less Is More for Alignment,” in *Advances in Neural Information Processing Systems 36*, 2023. doi: 10.52202/075280-2400.
- [4] H. Touvron, L. Martin, K. Stone, and others, “Llama 2: Open Foundation and Fine-Tuned Chat Models,” *arXiv preprint arXiv:2307.09288*, 2023, doi: 10.48550/arXiv.2307.09288.
- [5] Hugging Face, “Chat Templates.” 2026. [Online]. Available: https://huggingface.co/docs/transformers/chat_templating
- [6] S. Longpre *et al.*, “The FLAN Collection: Designing Data and Methods for Effective Instruction Tuning,” in *Proceedings of the 40th International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 22631–22648. [Online]. Available: <https://proceedings.mlr.press/v202/longpre23a.html>
- [7] Y. Wang *et al.*, “Self-Instruct: Aligning Language Models with Self-Generated Instructions,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, 2023, pp. 13484–13508. doi: 10.18653/v1/2023.acl-long.754.
- [8] E. J. Hu *et al.*, “LoRA: Low-Rank Adaptation of Large Language Models,” in *International Conference on Learning Representations*, 2022. [Online]. Available: <https://openreview.net/forum?id=nZeVKeeFYf9>